

Name: Praisel Richard Ekpenyong

Email: p.ekpenyong8162@miva.edu.ng

Department: Cybersecurity

Matric Number: 2025/A/CYB/0205

Source Files: [https://github.com/praisel-ekpenyong/COS201\\_Student\\_Course\\_Management\\_System](https://github.com/praisel-ekpenyong/COS201_Student_Course_Management_System)

## Source Code for Student Course Management System

- Main.java
- CourseManager.java
- Course.java

### 1. Main.java

This section contains the main program entry point. It displays the menu, reads user choices, and calls the course manager to perform each selected operation.

```
import java.io.IOException;
import java.util.Locale;
import java.util.Scanner;

// Console-based entry point for the Student Course Management System.
public class Main {
    private static final String DEFAULT_FILENAME = "courses.txt";
    private static final Scanner scanner = new Scanner(System.in);
    private static final CourseManager manager = new CourseManager();

    public static void main(String[] args) {
        printBanner();
        runMenu(0);
        scanner.close();
    }

    // Recursively displays the menu until the user chooses to exit.
    private static void runMenu(int depth) {
        if (depth > 500) {
            System.out.println("\nSession limit reached. Goodbye!");
            return;
        }

        printMenu();
        int choice = getValidChoice();

        if (handleChoice(choice)) {
            System.out.println();
            runMenu(depth + 1);
        }
    }

    private static boolean handleChoice(int choice) {
        switch (choice) {
            case 1:
                handleAddCourse();
                return true;
        }
    }
}
```

```

        case 2:
            handleViewCourses();
            return true;
        case 3:
            handleSearchCourse();
            return true;
        case 4:
            handleComputeUnits();
            return true;
        case 5:
            handleSave();
            return true;
        case 6:
            handleLoad();
            return true;
        case 7:
            System.out.println("\nThank you for using the Course Management System.
Goodbye!");
            return false;
        default:
            System.out.println("Invalid choice. Please choose 1 - 7.");
            return true;
    }
}

private static void printBanner() {
    System.out.println("=====");
    System.out.println("    STUDENT COURSE MANAGEMENT SYSTEM    ");
    System.out.println("=====");
    System.out.println();
}

private static void printMenu() {
    System.out.println("----- MENU -----");
    System.out.println(" 1. Add Course");
    System.out.println(" 2. View All Courses");
    System.out.println(" 3. Search Course by Code");
    System.out.println(" 4. Compute Total Units");
    System.out.println(" 5. Save to File");
    System.out.println(" 6. Load from File");
    System.out.println(" 7. Exit Program");
    System.out.println("-----");
}

// Validates menu input using a loop and exception handling.
private static int getValidChoice() {
    while (true) {
        System.out.print("\nEnter your choice (1-7): ");
        String input = scanner.nextLine().trim();

        try {
            int choice = Integer.parseInt(input);
            if (choice >= 1 && choice <= 7) {
                return choice;
            }
            System.out.println("Error: Please enter a number between 1 and 7.");
        } catch (NumberFormatException ex) {
            System.out.println("Error: Invalid input. Please enter a valid number.");
        }
    }
}

private static String getNonEmptyString(String prompt) {
    while (true) {
        System.out.print(prompt);
        String value = scanner.nextLine().trim();
        if (!value.isEmpty()) {

```

```

        return value;
    }
    System.out.println("Error: Input cannot be empty. Please try again.");
}

private static int getPositiveInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        String input = scanner.nextLine().trim();

        if (input.isEmpty()) {
            System.out.println("Error: Value cannot be empty.");
            continue;
        }

        try {
            int value = Integer.parseInt(input);
            if (value <= 0) {
                System.out.println("Error: Units must be greater than zero.");
                continue;
            }
            return value;
        } catch (NumberFormatException ex) {
            System.out.println("Error: Please enter a valid whole number.");
        }
    }
}

private static void handleAddCourse() {
    System.out.println("\n--- Add Course ---");
    try {
        String code = getNonEmptyString("Enter course code (e.g., COS201): ");
        String title = getNonEmptyString("Enter course title: ");
        int units = getPositiveInt("Enter credit units: ");
        manager.addCourse(code, title, units);
        System.out.println("\nCourse '" + code.toUpperCase(Locale.ROOT) + "' added successfully!");
    } catch (IllegalArgumentException ex) {
        System.out.println("\nError: " + ex.getMessage());
    }
}

private static void handleViewCourses() {
    System.out.println("\n--- All Courses ---");
    manager.displayAll();
}

private static void handleSearchCourse() {
    System.out.println("\n--- Search Course by Code ---");
    String code = getNonEmptyString("Enter course code to search: ");
    Course result = manager.searchByCode(code);

    if (result != null) {
        System.out.println("\nCourse found:\n  " + result);
    } else {
        System.out.println("\nNo course found with code '" + code.toUpperCase(Locale.ROOT) + "'.");
    }
}

private static void handleComputeUnits() {
    System.out.println("\n--- Total Credit Units ---");
    long total = manager.computeTotalUnits();
    int count = manager.getCourseCount();

    if (count == 0) {
        System.out.println("No courses recorded. Total units: 0");
    }
}

```

```

    } else {
        System.out.println("Total courses: " + count);
        System.out.println("Total credit units: " + total);
    }
}

private static void handleSave() {
    System.out.println("\n--- Save to File ---");
    System.out.print("Enter filename [" + DEFAULT_FILENAME + "]: ");
    String filename = scanner.nextLine().trim();
    if (filename.isEmpty()) {
        filename = DEFAULT_FILENAME;
    }

    try {
        manager.saveToFile(filename);
        System.out.println("\nCourses saved successfully to '" + filename + "'.");
    } catch (IOException ex) {
        System.out.println("\nError: " + ex.getMessage());
    }
}

private static void handleLoad() {
    System.out.println("\n--- Load from File ---");
    System.out.print("Enter filename [" + DEFAULT_FILENAME + "]: ");
    String filename = scanner.nextLine().trim();
    if (filename.isEmpty()) {
        filename = DEFAULT_FILENAME;
    }

    try {
        manager.loadFromFile(filename);
        System.out.println("\nCourses loaded successfully from '" + filename + "'.");
        System.out.println("Loaded " + manager.getCourseCount() + " course(s).");
    } catch (IOException ex) {
        System.out.println("\nError: " + ex.getMessage());
    }
}
}

```

## 2. CourseManager.java

This section contains the management logic for the system. It stores courses, adds new records, searches and updates course details, deletes courses, and saves course data to the text file.

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

// Manages the course list and all core operations.
public class CourseManager {
    private final List<Course> courses;

    public CourseManager() {
        courses = new ArrayList<>();
    }

    public int getCourseCount() {
        return courses.size();
    }

    public void addCourse(String code, String title, int units) throws IllegalArgumentException {
        Course newCourse = new Course(code, title, units);

        for (Course course : courses) {
            if (course.getCode().equals(newCourse.getCode())) {
                throw new IllegalArgumentException(
                    "Course with code '" + newCourse.getCode() + "' already exists.");
            }
        }

        courses.add(newCourse);
    }

    public Course searchByCode(String code) {
        if (code == null || code.trim().isEmpty()) {
            return null;
        }
        return recursiveSearch(code.trim().toUpperCase(Locale.ROOT), 0);
    }

    // Base case: return null when the end of the list is reached.
    private Course recursiveSearch(String code, int index) {
        if (index >= courses.size()) {
            return null;
        }
        if (courses.get(index).getCode().equals(code)) {
            return courses.get(index);
        }
        return recursiveSearch(code, index + 1);
    }

    public long computeTotalUnits() {
        return recursiveSumUnits(courses.size() - 1);
    }

    // Base case: return 0 when index goes below the first course.
    private long recursiveSumUnits(int index) {
        if (index < 0) {
            return 0L;
        }
    }
}
```

```

    }
    return courses.get(index).getUnits() + recursiveSumUnits(index - 1);
}

public void displayAll() {
    if (courses.isEmpty()) {
        System.out.println("No courses recorded yet.");
        return;
    }

    System.out.println();
    System.out.println("+-----+-----+-----+");
    System.out.println("| Code      | Title                                | Units|");
    System.out.println("+-----+-----+-----+");

    for (Course course : courses) {
        System.out.println(course);
    }

    System.out.println("+-----+-----+-----+");
    System.out.println("Total courses: " + courses.size());
}

// Saves courses in CODE|TITLE|UNITS format.
public void saveToFile(String filename) throws IOException {
    try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
        for (Course course : courses) {
            writer.write(course.toFileLine());
            writer.newLine();
        }
    } catch (IOException ex) {
        throw new IOException("Failed to save courses to '" + filename + "': " +
ex.getMessage(), ex);
    }
}

// Loads into a temporary list so invalid files do not erase current data.
public void loadFromFile(String filename) throws IOException {
    List<Course> loadedCourses = new ArrayList<>();

    try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
        String line;
        int lineNumber = 0;

        while ((line = reader.readLine()) != null) {
            lineNumber++;
            line = line.trim();
            if (line.isEmpty()) {
                continue;
            }

            try {
                Course course = Course.fromFileLine(line);
                if (containsCode(loadedCourses, course.getCode())) {
                    throw new IllegalArgumentException(
                        "Duplicate course code '" + course.getCode() + "'.");
                }
                loadedCourses.add(course);
            } catch (IllegalArgumentException ex) {
                throw new IOException("Error on line " + lineNumber + ": " + ex.getMessage(),
ex);
            }
        }
    } catch (FileNotFoundException ex) {
        throw new FileNotFoundException("File '" + filename + "' not found.");
    }

    courses.clear();
}

```

```
        courses.addAll(loadedCourses);
    }

    private static boolean containsCode(List<Course> courseList, String code) {
        for (Course course : courseList) {
            if (course.getCode().equals(code)) {
                return true;
            }
        }
        return false;
    }
}
```

### 3. Course.java

This section defines the Course object. It holds each course's code, title, credit unit, and lecturer information, and provides methods for accessing and updating those values.

```
import java.util.Locale;

// Represents a validated, immutable course record.
public class Course {
    private static final String FILE_DELIMITER = "|";

    private final String code;
    private final String title;
    private final int units;

    public Course(String code, String title, int units) {
        this.code = normalizeRequired(code, "Course code").toUpperCase(Locale.ROOT);
        this.title = normalizeRequired(title, "Course title");

        if (this.code.contains(FILE_DELIMITER) || this.title.contains(FILE_DELIMITER)) {
            throw new IllegalArgumentException("Course code and title cannot contain '|'.");
        }
        if (units <= 0) {
            throw new IllegalArgumentException("Units must be a positive integer.");
        }

        this.units = units;
    }

    private static String normalizeRequired(String value, String fieldName) {
        if (value == null || value.trim().isEmpty()) {
            throw new IllegalArgumentException(fieldName + " cannot be empty.");
        }
        return value.trim();
    }

    public String getCode() {
        return code;
    }

    public String getTitle() {
        return title;
    }

    public int getUnits() {
        return units;
    }

    public String toFileLine() {
        return code + "|" + title + "|" + units;
    }

    public static Course fromFileLine(String line) throws IllegalArgumentException {
        if (line == null) {
            throw new IllegalArgumentException("Course record cannot be null.");
        }

        String[] parts = line.split("\\|", 3);
        if (parts.length != 3) {
            throw new IllegalArgumentException("Invalid course record: " + line);
        }

        try {
            int units = Integer.parseInt(parts[2].trim());
            return new Course(parts[0], parts[1], units);
        } catch (NumberFormatException ex) {
        }
    }
}
```



```
        throw new IllegalArgumentException("Invalid units in record: " + line, ex);
    }

    @Override
    public String toString() {
        return String.format("| %-10s | %-30s | %3d |", code, title, units);
    }
}
```